

OUTLIERS

Going and Looking

Layer 2 of 5 · Four jobs that go out,
and bring people back.

EST. MMXXVI · OUTLIERS ACCELERATOR

Walking through the door

Four jobs, given in the order of what it costs you to get one wrong. The first three cannot get you into trouble at all. The fourth is where you find out the whole set works, and it is chosen because being wrong there costs nothing.

WHAT THE LAST LAYER LEFT YOU WITH

You have a browser that can go outside and nothing that goes anywhere. The doorman stands at a door nobody has walked through, and the only way to know it works is to send something through.

WHAT THIS LAYER BUILDS

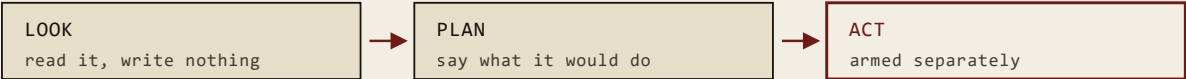
Four jobs. One finds people. One undoes requests you sent yourself and forgot. One asks somebody to connect. One works out who said yes.

Everything they find goes into the records system you already built, as events, through the resolver you already wrote. Nothing here keeps its own list of people, because a second list of people is a second answer to who somebody is.

IN THE ORDER OF WHAT BEING WRONG COSTS YOU



AND EVERY ONE OF THEM RUNS THE SAME THREE WAYS



FOUR JOBS, THREE MODES EACH, ONE ORDER.

How it works

One shape, four times. You will use the first mode far more than you expect to.

LOOK, THEN PLAN, THEN ACT

Every job here runs three ways. **Look** opens the page, reads it, and writes nothing anywhere. **Plan** is what you get by default: it works out exactly what it would do and tells you, without doing any of it. **Act** has to be asked for separately.

The reason the first mode exists is not caution. It is that a page you have never read is a page you cannot write a job against, and a page that changed overnight will fail in a way that reads like your code being broken. Looking first tells you which of the two you have.

THE FOUR JOBS, AND WHY THIS ORDER

- **Load a file.** An export you already own, sitting on your own machine. No sign-in, no reading of anything, no limits touched. Start here.
- **Find people.** Your own connections, a search, or whoever reacted to a post. Reading only. The last of those is the strongest one you have if your network is thin, because those people raised a hand on the subject themselves.
- **Undo your own requests.** Cancels invitations you sent and forgot. Nobody else is touched, which is exactly why it is the first one you run for real.
- **Ask to connect.** The first job that reaches another person. It carries none of your words, which is why it belongs here and a message does not.

WHERE WHAT YOU FIND ENDS UP

Every job hands what it found to the event log in your records system, through the same resolver every collector you built already uses. Somebody it cannot place gets a provisional record of their own and is never merged into anybody. A merge decides whose history is right, and that decision stays yours.

THE ONE THAT WILL SURPRISE YOU

A free search allowance is spent by **searching**, not by what comes back. Cross it and you are cut to about three results a query until the first of the month. So one wide search read deep is safe where fifty narrow ones are not, and the job stops and says so rather than spending the rest of your allowance finding out.

Which means the search is the one job whose **look** mode is not free: a search costs a search whether or not you keep the results. It is counted like any other, and the job says so before it opens anything. The other three cost nothing to look at.

Build it

One command. It asks you a few questions so what you get is yours, then does the work.

```
git clone https://github.com/OUTLIERS-ai/outliers-gather-02-basics
cd outliers-gather-02-basics
python install.py
```

This needs Layer 1 already installed, and finds it by itself. Nothing reaches the outside world during installation.

WHAT IT ASKS YOU, AND WHY

QUESTION	WHAT IT CHANGES
After how many days is an unanswered request stale?	How long a request waits before the undo job will take it back. Nothing is taken back before then.
How many searches a month should it assume you have?	Your allowance, which the search job counts down and stops at. Guessing low costs you nothing; guessing high spends it without warning.
At most, how many people should one reading run bring back?	A ceiling on a single run, so a long list cannot turn into an afternoon of reading nobody asked for.

WHAT YOU END UP WITH

- Four jobs, each of which will read a page and write nothing if you ask it to.
- Everything they find flowing into the event log you already had.
- A record of what you did, kept separately from what is true about the person.

Now use it

Start with the one that cannot go wrong.

In the **_engine** folder inside your CRM:

```
python gather.py find export your-file.csv
```

It reads the file, tells you how many people it found, how many you already had, and how many it could not place. Nothing has been written yet. Run it again with **--commit** and those people are in your records.

Now the first one that touches the outside world, in its safest mode:

```
python gather.py undo --probe
```

It opens your sent invitations, reads them, and writes nothing. If it finds none, that is a clean result rather than a failure.

IF YOU ALREADY HAD SOMETHING LIKE THIS

A test rather than an instruction. Take the last twenty people who came into your records and answer where each one came from. If you cannot, nothing you measure later will mean anything, because you will not know which source produced it.

WHAT THIS LAYER LEAVES UNSOLVED

All four work, and you typed every one of them yourself. Worse, you can only do this for one website. Nothing here tells you how any of it was arrived at, so the moment you want a different source, or a different platform, you are back to guessing.

NEXT: LAYER 3 • HOW THIS WAS BUILT

The method behind layers 1 and 2, in enough detail to rebuild them yourself or point them at something we never touched.

WORDS USED IN THIS GUIDE

Probe	A run that reads a page and writes nothing anywhere. It still asks the doorman, because opening a page is reaching outside.
Commit	Asking for a job to actually do what it says it would do.
Export	A file of your own data that a site will give you if you ask it.
Event	One line in your records saying something happened, and when.